Common Table Expression~CTE

What is a CTE?

CTE stands for **Common Table Expression**.

A CTE is a **temporary result set** that you define at the beginning of a SQL query using the keyword **`WITH`**.
You can then use this temporary table in the main query.

It makes **complex queries easier to read and organize**.

Basic Syntax

```
WITH cte_name AS (
    SELECT ...
)
SELECT *
FROM cte_name;
```

- `cte_name` → temporary table name
- The CTE exists **only during that query**

Example Using Your `sales_data` Dataset

1. CTE Example: Total Revenue per Product

First we calculate revenue per product, then select from it.

```
WITH product_sales AS (
    SELECT product, SUM(revenue) AS total_revenue
    FROM sales_data
    GROUP BY product
)
SELECT *
FROM product_sales
ORDER BY total_revenue DESC;
```

What happens

Step 1 (CTE):

product	total_revenue
Laptop	500000
Camera	480000
Tablet	420000

Step 2:

The main query selects from this **temporary table**.

---

2. CTE Example: Top Product in Each Region

```
WITH ranked_products AS (
    SELECT
        region,
        product,
        SUM(revenue) AS total_revenue,
        RANK() OVER (PARTITION BY region ORDER BY SUM(revenue) DESC) AS rnk
    FROM sales_data
    GROUP BY region, product
)
SELECT region, product, total_revenue
FROM ranked_products
WHERE rnk = 1;
```

Purpose

Find the **highest revenue product in each region**.

---

3. CTE Example: Customer Spending Analysis

Find customers who spent **more than average spending**.

```
WITH customer_spending AS (
    SELECT customer, SUM(revenue) AS total_spent
    FROM sales_data
    GROUP BY customer
)
SELECT *
FROM customer_spending
WHERE total_spent > (
    SELECT AVG(total_spent)
    FROM customer_spending
);
```

Logic

1. CTE calculates **total spending per customer**
2. Main query filters customers **above average spending**

---

4. CTE Example: Daily Sales with Running Total

Common Table Expression~CTE

```
WITH daily_sales AS (
    SELECT order_date, SUM(revenue) AS daily_revenue
    FROM sales_data
    GROUP BY order_date
)
SELECT
order_date,
daily_revenue,
SUM(daily_revenue) OVER (ORDER BY order_date) AS cumulative_sales
FROM daily_sales;
```

This shows **cumulative sales over time**.

CTE vs Subquery

Feature	CTE	Subquery
Readability	Easier for complex queries	Harder to read
Reuse	Can reuse multiple times	Usually used once
Structure	Written at the top	Inside query

Simple Way to Remember

CTE = Temporary table created inside a query

Example idea:

```
WITH temporary_table AS (...)
SELECT ...
FROM temporary_table
```

💡 Since you are practicing **SQL for data analytics**, a very important next concept is **Recursive CTE** (used in hierarchical data and graph analysis).

If you want, I can also show **5 advanced CTE interview problems using your sales dataset**.

give a note about CTE and why we use CTE?

Common Table Expression~CTE

Here is a **clear note about CTE (Common Table Expression)** you can use for study or exam.

Common Table Expression (CTE)

Definition

A **Common Table Expression (CTE)** is a temporary result set in SQL that is defined using the **`WITH`** clause and can be referenced within a **SELECT, INSERT, UPDATE, or DELETE** statement.

A CTE exists only during the execution of a single query and is not stored permanently in the database.

Syntax of CTE

```
WITH cte_name AS (
    SELECT column1, column2
    FROM table_name
)
SELECT *
FROM cte_name;
```

- `WITH` → keyword used to define the CTE
- `cte_name` → temporary table name
- The CTE is used in the main query just like a normal table.

Example Using Sales Data

```
WITH product_sales AS (
    SELECT product, SUM(revenue) AS total_revenue
    FROM sales_data
    GROUP BY product
)
SELECT *
FROM product_sales;
```

Explanation

1. The CTE `product_sales` calculates total revenue for each product.
2. The main query retrieves the result from this temporary table.

Common Table Expression~CTE

Why We Use CTE

CTEs are used mainly to make SQL queries **clear, organized, and easier to maintain**.

1. Improve Readability

Complex queries become easier to understand.

Example:

Instead of writing a long nested query, we separate logic into steps.

---

2. Simplify Complex Queries

CTEs help break a large query into smaller logical parts.

---

3. Reuse Query Results

A CTE can be referenced multiple times within the same query.

---

4. Replace Nested Subqueries

CTEs often make queries easier than deeply nested subqueries.

---

5. Work with Window Functions and Rankings

CTEs are commonly used with functions like:

- `RANK()`
- `ROW_NUMBER()`
- `LAG()`
- `LEAD()`

Example with Ranking

Common Table Expression~CTE

```
WITH ranked_products AS (
    SELECT
        region,
        product,
        SUM(revenue) AS total_revenue,
        RANK() OVER (PARTITION BY region ORDER BY SUM(revenue) DESC) AS
rank_no
    FROM sales_data
    GROUP BY region, product
)
SELECT *
FROM ranked_products
WHERE rank_no <= 3;
```

This query finds the **top 3 products by revenue in each region**.

Advantages of CTE

- Improves query readability
- Makes complex queries easier to manage
- Helps organize step-by-step calculations
- Useful for advanced data analysis
- Can be used with recursive queries